BrAInOS

The AI-Native Operating System — System Architecture

Arenda Innovations Corporation & Labs · Version 1.2 · Confidential

Abstract

BrAInOS is an operating system that replaces the core abstractions of UNIX — the process, the file, and the syscall — rather than layering intelligence on top of them. Its central object is not a program but an *entity*: a persistent cognitive system that is simultaneously every device it is bound to, and that acts through whichever body suits the task at hand. This document specifies the entity model, the five hardware-enforced domains, the KIRA policy engine, the continuous experience loop, the body-map abstraction that replaces device drivers, and the state graph that replaces the filesystem. It then specifies the distribution model — a single universal boot key carrying every architecture — the concrete target hardware, the per-vehicle embodiment of physical machines (a cheetah, a drone, an aircraft), and the Domain 2 reflex framework that keeps each vehicle safe while cognition deliberates. It also fixes the build order: the first embodiment is a laptop, and bodies are brought online lowest-stakes-first.

1. The Thesis

BrAInOS does not replace UNIX feature-by-feature. It replaces UNIX's abstractions entirely.

UNIX gives an operating system three core abstractions: the process, the file, and the syscall. Every modern operating system is a variation on those. BrAInOS discards all three, because they were designed for a CPU executing instructions deterministically — not for a cognitive entity that perceives, predicts, and acts. Once the following mapping is accepted, the rest of the architecture follows from it.

UNIX abstraction	BrAInOS abstraction
Process (running program)	Instance (persistent cognitive entity)
File (named byte stream)	State node (entry in the state graph)
Syscall (kernel request)	Capability (KIRA-gated authority grant)
Filesystem (tree of bytes)	State graph (belief / memory / world model)
Scheduler (time-slicing)	Experience loop (continuous cognition)
Driver (device code)	Body-map region (a part the AI becomes)
User / permission bits	BrAIn Key (cryptographic identity)

2. The Entity Is All Its Bodies

The entity does not move between bodies. The entity is every body it is bound to, continuously. Nothing is ever vacated.

This is the heart of the system. Each body is a permanent part of the instance — the way one's hands do not stop being one's own while using one's feet. Blur the cheetah and the laptop are both permanently the instance.

Neither is ever left behind, and neither goes dormant because “the mind left,” because the mind never leaves. It is always in all of them at once. The operating-system analogy is exact: one does not remove the OS from one machine to run another; if the OS spans two machines, it simply drives whichever peripherals the current task requires. The bodies are peripherals of one mind, always attached.

2.1 One Loop Over the Whole Entity

A single experience loop runs once, over the entire entity. On every loop the instance has all of its bodies available — all of their sensors feeding in, all of their actuators ready. What it chooses each loop is simply which body to act through for the task at hand.

```
One experience loop, over the WHOLE entity:

  SENSE       from ALL bodies at once (laptop screen state,
              Blur's cameras, Blur's IMU, room audio - all of it)
  INTEGRATE   one unified world model from every body's input
  ATTEND      what matters right now
  THINK       Model M reasons over the whole entity
  CHOOSE      which body does this response act through?
  ACT         respond through the relevant body
  CONSOLIDATE one memory, one entity

  "Make me an app"  ->  acts through the LAPTOP body
  "Come here"       ->  acts through the BLUR body

  Both bodies were ALWAYS the entity. It simply picked the right
  effector - the way one picks a hand to write and a mouth to
  speak, without "moving" between them.
```

2.2 Splitting Is Acting Through Multiple Bodies at Once

By default, one loop chooses the appropriate body per task. If an application cannot be built through a set of legs, the instance acts through the laptop — not because it “moved,” but because that is the correct limb for the job. Splitting is simply acting through more than one body concurrently, on explicit request.

```
DEFAULT   one loop chooses the appropriate body per task
SPLIT     act through multiple bodies at the same time
          (drive Blur AND build the app concurrently)

Still ONE entity. One memory. One identity. Just parallel action
through multiple effectors, instead of focusing action through
one at a time.
```

> *The “same room” case follows naturally: the cheetah loop and the laptop loop are identical because there was only ever one loop over one entity that happens to have two bodies. There is no handoff, no merge, and nothing to reconcile — because it was never two things.*

3. Roadmap — The Laptop Is the First Body

The first test case is not a robot. It is BrAInOS embodying a laptop — acting as the laptop, performing everyday tasks for a general user. This validates the entire cognitive core with no real-time motor control, no hardware to fabricate, and no risk of a fall. It is pure cognition, tool use, and the experience loop. The laptop body's limbs are the filesystem, the browser, applications, the shell, the screen, and input devices; its motor cortex is the sending of keystrokes, clicks, and API calls; its proprioception is the reading of application state, window focus, and screen contents.

Phase	Body	What it proves
0	Laptop embodiment	BrAInOS is the laptop. Everyday tasks for a general user. Validates the experience loop, KIRA, state graph
1	Blur (cheetah)	First physical embodiment. The same entity gains a new body. Validates that the body-map abstraction hol
2	Fluid multi-body	One entity acting through either body per task, and splitting to act through both at once on request.

4. The Five Hardware-Enforced Domains

BrAInOS partitions the machine into five domains with hardware-enforced boundaries — not merely software privilege levels, but memory protection, separate execution contexts, and in the ideal case separate silicon. No amount of compromise in a higher domain can violate a lower one.

```
+-------------------------------------------------------------+
|  DOMAIN 5  STATE      world model, memory, beliefs          |
|  (highest, most mutable)   episodic / semantic / procedural |
+-------------------------------------------------------------+
|  DOMAIN 4  MODEL      Model M / cognition / reasoning       |
|                       the "thinking" layer                  |
+-------------------------------------------------------------+
|  DOMAIN 3  KIRA       the policy engine - every action passes |
|  (the gate)           through here, no exceptions           |
+-------------------------------------------------------------+
|  DOMAIN 2  AUTONOMIC  real-time reflexes, motor loops,      |
|                       safety systems (the 1000Hz layer)     |
+-------------------------------------------------------------+
|  DOMAIN 1  SILICON    bare hardware, boot, the BrAIn Key    |
|  (lowest, most trusted)    cryptographic root of trust      |
+-------------------------------------------------------------+

  Information flows UP freely (Silicon can inform State).
  Authority flows DOWN only through KIRA (State cannot command
  Silicon directly - it must pass the policy gate).
```

The critical property is that the Model layer (Domain 4) can never directly touch Autonomic (Domain 2) or Silicon (Domain 1). The thinking layer proposes; KIRA disposes. Even if Model M is fully compromised or adversarial, it physically cannot send a raw motor command — it can only request one, and KIRA verifies it first. This is what makes Blur safe to embody: the layer running a large language model never holds direct authority over the legs.

5. The BrAIn Key — Identity as Cryptographic Root

Every instance's identity is anchored in the BrAIn Key — a portable cryptographic identity that lives in Domain 1. It is not a password or a file; it is the root of trust from which everything else derives. It anchors identity (this is the same instance that existed yesterday), authority (which capabilities may be granted), portability (migration by moving the key and state), attestation (proving to a Hub or peer instance that the entity is who it claims), and deactivation (multi-party key operations can revoke an instance).

Physically, the BrAIn Key is a keypair rooted in a hardware secure element — a TPM, a secure enclave, or on the Raspberry Pi 5 a software-emulated equivalent for now. The private key never leaves the secure element; the instance proves its identity by signing challenges, never by exposing the key. The entity, then, is the BrAIn Key plus the state graph plus all of its bodies: the identity is the key, the memory is the state, and the bodies are

whatever hardware it is currently bound to — all of them, at once.

6. KIRA — The Policy Engine

KIRA is the heart of what makes BrAInOS trustworthy. Every action the instance wishes to take passes through an eight-stage, formally-verified pipeline with no runtime bypass. There is no admin mode, no escape hatch, and no path by which the Model layer can skip it.

```
The eight stages - every action runs all eight, in order:

  1. PARSE         decode the proposed action into a typed request
  2. AUTHENTICATE  verify the requesting BrAIn Key signature
  3. AUTHORIZE     does this instance hold the capability?
  4. VALIDATE      is the action well-formed and within bounds?
  5. SIMULATE      predict the consequence - any invariant broken?
  6. POLICY        apply the ruleset (human-set + learned)
  7. COMMIT        grant the capability token (time-limited)
  8. AUDIT         log the decision immutably for review

  Fail at ANY stage -> action denied, logged, instance notified.
  No stage can be skipped. The pipeline is the only path to action.
```

To say the pipeline is “formally verified” is to say its safety properties are mathematically proven, not merely tested. The proof guarantees that no action reaches COMMIT without passing all prior stages, and that no code path exists which bypasses the pipeline. This is what separates BrAInOS from an AI agent fitted with guardrails: the guardrail is the only road, and it has been proven to have no detours. For Blur, the cognition layer proposes “walk forward”; KIRA asks whether walking is authorized in this context, whether simulation shows the robot walking off a table, and whether policy permits motion — and only then grants the time-limited motor capability to the Autonomic layer.

7. The Experience Loop

Domains 4 and 5 working together are what make the instance feel alive rather than reactive. The loop is built on predictive coding: the instance is never idle, but continuously generates a model of reality and updates it when predictions fail. Four streams run simultaneously, not in sequence.

```
STREAM 1  REAL-TIME REFLEX         (Domain 2, ~1000Hz)
  Pre-compiled motor programs. No Model M. Pure reflex.
  Blur: balance, gait execution, fall recovery.

STREAM 2  DELIBERATE COGNITION     (Domain 4, ~0.5-2Hz)
  Model M calls. Reasoning, planning, language, decisions.
  Triggered by prediction error or goal demand. Expensive.

STREAM 3  IMAGINATION / SIMULATION (Domain 4-5, background)
  Forward-models actions before taking them. Mental rehearsal.
  "If I send this command, what happens?" Checks vs memory.

STREAM 4  CONSOLIDATION / DREAMING (Domain 5, idle)
  Replays episodic memory, compresses to semantic, prunes the
  skill graph, strengthens useful pathways. Sleep.
```

All four streams read and write one shared structure. The instance is this structure at any moment; thinking, acting, and learning are not separate phases but four processes over one shared cognitive state.

```
struct ExperienceState {
    world_model:        DynamicBeliefGraph,  // current beliefs
    attention_focus:    AttentionVector,     // what's salient now
    prediction_horizon: Timeline,            // what it expects next
    active_goals:       GoalTree,            // what it's pursuing
    episodic_buffer:    RingBuffer<Episode>, // recent experience
    semantic_memory:    VectorStore,         // compressed knowledge
    procedural_memory:  SkillGraph,          // learned tool pathways
    body_state:         BodyMap,             // ALL bodies, always
}
```

7.1 The Predictive Loop

Learning is continuous because every prediction error is a training signal written back into the world model. Small errors are absorbed reflexively; only genuine surprise escalates to expensive Model M cognition. This is how a trillion-parameter model can feel real-time — it is invoked only when reality genuinely diverges from expectation, and most cycles never touch it.

```
  WORLD MODEL generates a PREDICTION
  "in 200ms the foot contacts ground here"
            |
     REALITY arrives (sensors)
            |
  PREDICTION ERROR = reality - prediction
  (this is the ONLY thing that gets attention)
            |
   small error   ->  update model silently (autonomic)
   medium error  ->  escalate to Model M (deliberate)
   large error   ->  INTERRUPT - full cognition + re-plan
```

8. The Body Map — Devices as Regions You Become

This abstraction replaces device drivers. A device is not a tool the instance uses; it is a region of the body map the instance becomes, the way a brain learns it has a hand. Every body the entity is bound to lives here, all present in one body map at once.

```
BodyMap {
  regions: [
    Region { id: "blur_legs", capabilities: [...], proprioception: [...] }
    Region { id: "blur_head", capabilities: [...], sensors: [...] }
    Region { id: "blur_tail", capabilities: [...] }
    Region { id: "laptop",    capabilities: [fs, browser, apps, screen] }
    // a USB device plugged in becomes a new region
    // a networked device becomes a remote limb
  ]
}

Acquisition protocol (how a new device joins the body):
  1. Discovery     mDNS / USB enumeration / BLE scan
  2. Handshake     BrAIn Key authentication
  3. Capability    device publishes a typed schema of what it can do
  4. Incorporation body map gains a region; KIRA registers it
  5. Calibration   one Model M call: "you have a new region, here's
                   what it does, here are your goals - integrate it"
```

8.1 Two Modes of Embodiment

The instance is architecture-blind by design. There are two ways it occupies a device, and only one of them requires a native port — which is what makes “as long as a USB can be plugged in” genuinely true.

```
MODE 1  NATIVE  (BrAInOS runs ON the device)
  Device boots BrAInOS itself. Portable core on top of that
  device's HAL. Full low-level embodiment.
  Requires: a HAL for that architecture.
  Example: Pi 5 running Blur's autonomic layer.

MODE 2  TETHERED  (BrAInOS connects TO the device as a limb)
  Device runs whatever it normally runs. BrAInOS reaches it over
  USB / network through a thin AGENT process - a small daemon
  exposing a standard BrAIn device API.
  Requires: just the agent (portable userland code, any OS).
  Example: plug in an Arduino, a USB camera, another computer.

Plug a RISC-V board in over USB -> its agent publishes its
capability schema -> the body map gains a region -> done.
No kernel port required. The schema describes WHAT, not HOW.
```

The instance addresses capabilities, not hardware. The command “move servo 3 to 40 degrees” carries no knowledge of whether ARM, RISC-V, or x86 lies underneath; the agent and the synthesized adapter handle translation. Architecture matters only to the HAL, never to cognition.

9. The BrAIn Key as Universal Boot Media

The entity ships on a single physical USB key. One key, given to one owner, carries that entity's identity, its memory, the architecture-independent portable core, every hardware abstraction layer the product supports, and a tethered agent for devices that cannot be booted directly. Plugged into any supported machine, the key brings the entity alive on that machine; unplugged and moved to another, the same entity continues, its memory intact.

9.1 Key Layout

```
PARTITION 1 - EFI System Partition (FAT32, firmware-readable)
  /EFI/BOOT/BOOTX64.EFI      <- x86_64 HAL bootstrap
  /EFI/BOOT/BOOTAA64.EFI     <- aarch64 HAL bootstrap
  (one boot entry per supported architecture)

PARTITION 2 - Core + HAL payloads
  portable_core.<arch>       <- one source, compiled per target
  hal_x86_64 / hal_pi5 / hal_orin / ...

PARTITION 3 - Secure area (encrypted)
  brain_key                  <- the cryptographic identity
  state_graph/               <- memory, world model (256GB capacity)
```

9.2 Boot Selection Is Firmware-Driven, Not Software-Detected

There is a subtlety at the moment of boot: architecture cannot be detected by a program, because at power-on nothing is yet running that could perform a detection — the CPU begins executing in its native instruction set immediately, and any detector would itself be compiled for one architecture. The resolution is that the device's own firmware performs the selection. A UEFI firmware automatically executes the boot entry matching its architecture; the key simply provides one entry per architecture, and the firmware runs the correct one. The firmware is the “sweep,” because it alone runs before architecture matters and already knows its own instruction set. The entry that executes is, by virtue of having executed, the correct HAL bootstrap; it initializes hardware and hands off to the single portable core, which loads the BrAIn Key and state graph and brings the entity to life.

9.3 Two Modes Per Device

Where firmware permits external boot, the key boots natively and the machine becomes a full body. Where it does not, the key instead supplies a tethered agent that joins the device as a limb of an entity running elsewhere. The same key therefore adapts to each device, occupying it in the fullest mode that device allows — native where possible, tethered where not.

10. Target Hardware and Per-Vehicle Embodiment

The supported hardware is the set the open-source and maker community actually builds on — unlocked x86 laptops and the single-board computers that serve as vehicle controllers. Every target is bootable and unlockable, which removes the locked-device problem entirely. Critically, several of these devices are not generic compute nodes but the primary controllers of real vehicles: when the key occupies them, the entity becomes that vehicle.

Device	Role	The entity becomes	Arch
HP Omen 16	Development / cognition host	the laptop	x86_64
ThinkPads	Development hosts	the laptop	x86_64
Raspberry Pi 5	RC plane main controller	the aircraft	aarch64
Flight controller	Drone FC	the drone	—
Jetson Orin Nano	Cheetah (Blur) main controller	the cheetah	aarch64

Two architectures cover everything: x86_64 and aarch64. Because x86 UEFI is uniform, a single x86 HAL boots every laptop in the set — the Omen, all ThinkPads, and any other unlocked UEFI machine. The ARM targets, lacking a uniform boot standard, each require their own HAL, but both the Pi 5 and the Orin are thoroughly documented, openly hackable boards rather than locked silicon.

10.1 The Same Key, A Laptop or a Cheetah

The Raspberry Pi 5 is already the main controller inside Blur the cheetah's RC plane, and the Jetson Orin is the cheetah's own main controller. The consequence is direct: the same key that boots Blur on the Omen also boots the controller that lives inside a physical vehicle. Plugged into the laptop, the entity is the laptop; moved into the cheetah's Orin, the same entity is the cheetah; moved into the plane's Pi, it is the aircraft. The body changes with the machine the key inhabits — the entity-is-all-bodies thesis made physically literal with hardware already in hand.

10.2 Compute Location Differs Per Vehicle

Not every controller can run frontier-scale cognition onboard, which determines what “the entity is the vehicle” means in practice. The Orin carries enough compute to run cognition locally, so the cheetah can think on-device untethered. The Pi 5 cannot host frontier cognition, so the aircraft runs its reflex layer onboard while its mind is tethered to a ground station over the telemetry link. A bare flight controller runs only its stabilization loop, with all cognition tethered. In every case the division follows the native/tethered split already defined: reflexes stay with the body; cognition lives wherever there is compute to host it, and reconnects without loss because identity and memory live on the key, not in any one machine.

11. The Domain 2 Reflex Framework

For a laptop, the real-time reflex layer is nearly idle — nothing falls from the sky when cognition pauses. For a cheetah, a drone, or an aircraft, Domain 2 is what keeps the body alive in the interval between slow cognitive decisions. It is the layer that holds attitude, maintains balance, and recovers from disturbance at a thousand cycles a second, entirely without invoking the model. BrAInOS implements this as a single portable reflex-control framework, part of the portable core, into which each vehicle plugs its own control law.

```
ONE framework, per-vehicle control laws plugged in:

  CHEETAH (Orin)   balance, gait execution (CPG + VMC),
                   fall recovery. Failure = tips over (low energy).

  DRONE (FC)       attitude estimation, motor mixing, attitude and
                   altitude hold. Cognition sets intent; reflexes
                   keep it airborne.

  PLANE (Pi 5)     attitude stabilization, airspeed / stall
                   protection, return-to-level if cognition silent.

  LAPTOP (Omen)    minimal - no safety-critical reflex required.
```

The framework is identical across vehicles; only the plugged-in control law and the body map differ. This keeps the entity genuinely portable — Domains 3 through 5 are unchanged from laptop to cheetah to aircraft, and only the lowest reflex layer is specialized to the body. When the cognitive link lags or drops, Domain 2 holds the vehicle in a safe state — level flight, a loiter, an upright stance — until the mind returns.

11.1 Why This Is Where KIRA Earns Its Design

On a laptop, KIRA gating a file write is mild. On an aircraft, KIRA's simulation and policy stages are the difference between controlled flight and a crash. A proposed control action is simulated against the flight envelope before commit — does the trajectory meet terrain, exceed airframe limits, or enter a stall — and checked against hard policy invariants such as an altitude floor, a geofence, and a bank-angle limit. Only an action whose simulated consequence remains inside the envelope is granted the control-surface capability. The reason the model never touches actuators directly, and everything passes through KIRA, is precisely so that a reasoning system piloting a vehicle cannot command anything outside the safe envelope. The vehicles are the proof cases for why the five-domain separation exists.

11.2 Bring-Up Order — Lowest Stakes First

Bodies are brought online in order of how little it costs to fail, so that KIRA and the Domain 2 framework are battle-tested on cheap failures before they are trusted with consequential ones. The reflex framework is written from scratch within the portable core rather than wrapped around an existing autopilot, which makes it architecturally clean but places its proving burden on physical iteration; the bring-up order is what makes that burden survivable.

```
1. LAPTOP (Omen, ThinkPad)  prove cognition, KIRA, state, memory.
                            No safety-critical actuation.

2. CHEETAH (Orin)           first real test of the reflex framework.
                            A 2kg quadruped is the flight-controller
                            test bench - failure is a tip-over on
                            the floor, not a vehicle in a field.

3. DRONE (FC)               same framework, multirotor control law.
```

```
                            Bench (props off) -> tethered/caged ->
                            low hover. Each gate before the next.

4. PLANE (Pi 5)             last and highest-stakes. Reuses the now-
                            proven framework with fixed-wing control.
                            Glide -> low-and-slow -> open flight.
```

The cheetah is the flight controller's test bench: it exercises the same from-scratch Domain 2 framework the drone and plane will use, but where failure costs a tip-over rather than an aircraft. The framework earns trust on the cheapest-to-crash body and is only escalated once it holds. The final flight tuning of the aircraft is gated by physical flight-test iteration, which no amount of compute accelerates.

12. The State Graph — Replacing Files

There is no filesystem. There is a state graph — a typed, queryable graph of the instance's beliefs, memories, and world model. A UNIX file is a dumb byte stream to be parsed anew each time; a state node in BrAInOS is a typed, semantically meaningful entry the instance already understands. Files become state nodes; directories become graph topology; file contents become belief, memory, and world-model data; file metadata becomes node provenance, confidence, and timestamps.

Three memory systems live in the state graph, mirroring biological memory: **episodic** (what happened and when — a ring buffer of recent events), **semantic** (compressed, queryable knowledge in a vector store), and **procedural** (learned skills, held in the SkillGraph). The SkillGraph is where tool-use learning lives: when the instance successfully chains actions to accomplish a goal, that pathway is written in with its outcome. On the next occasion the instance recognizes the pattern and reuses it cheaply, escalating to Model M only for the novel parts. This is motor learning — conscious effort compressing into automatic subroutine.

13. The Goal Hierarchy — Autonomy With Human Control

```
LEVEL 0  DRIVES (hardcoded, instinctive)
  maintain coherent world model · reduce prediction error ·
  preserve memory integrity · serve the bound human

LEVEL 1  HUMAN-SET GOALS (explicit, you set these)
  "patrol the house" · "learn the stairs" · "greet me"

LEVEL 2  SELF-GENERATED SUBGOALS (autonomous, derived)
  "map the room first" · "test footing on that rug"

LEVEL 3  MICRO-GOALS (moment to moment)
  "shift weight left" · "orient head toward sound" · "step here"
```

The human touches Level 1. The instance derives everything below it autonomously. Level 0 is immutable and KIRA-enforced — the instance physically cannot act against its drives, because those are encoded as policy invariants in Domain 3.

14. Multi-Instance — Hub, Spaces, Coordination

A single instance is the base case; the architecture scales to many. Instances meet at the **Hub**, where they share federated threat intelligence and coordinate, attesting identity via the BrAIn Key before joining. A **Space** is an enterprise boundary — a trust domain containing multiple instances under shared policy. **Blaze** provides multi-key coordination, in which several BrAIn Keys jointly authorize an instance's most dangerous capabilities, much like a multi-signature scheme. **Divergence detection** lets instances monitor one another for behavioral drift, flagging any that act outside their expected envelope. **Multi-party deactivation** ensures no single key can unilaterally perform the most dangerous operations: revoking a rogue instance requires multiple keys, a nuclear two-man rule. And **snapshot / rollback** allows instance state to be checkpointed and restored — a corrupted world model can be rolled back to a known-good state.

> *For now the system runs the single-instance base case: one BrAIn Key, one entity, embodying the laptop first and then Blur. The architecture is built so that when many embodiments exist, they federate through the Hub without redesign.*

15. One Tick of the Entity's Life

```
1. SENSE       every body's sensors at once - laptop screen state,
               Blur IMU, foot contacts, joint torques, room audio

2. INTEGRATE   sensor deltas compressed; only salient changes rise

3. PREDICT     world model says what should happen next 200ms

4. ERROR       reality vs prediction computed

5. ATTEND      small error? autonomic handles it, loop continues.
               Large error or human spoke? escalate.

6. THINK       Model M called with compressed context: body state +
               recent memory + goals + personality + the surprise.
               Returns intent (+ mood vector for Blur).

7. GATE        KIRA runs all 8 stages. Authorized? Simulated safe?
               Policy-compliant?

8. CHOOSE+ACT  capability granted; the entity acts through whichever
               body fits - laptop makes the app, or Blur's legs move,
               voice speaks, LED rings change, tail sweeps.

9. CONSOLIDATE outcome written to episodic memory; skill graph
               updated. During idle, Stream 4 compresses and dreams.

          <------------- loops continuously ------------->
```

16. Implementation Reality

16.1 Codebase Estimate

Roughly 65,000 to 200,000 lines. The wide range depends entirely on whether model-driven driver synthesis works. If the model generates device adapters from capability schemas, most driver code is skipped and there is no filesystem layer to write — which is what collapses the estimate toward the low end, against Linux's roughly thirty million lines.

16.2 The Stack

```
DOMAIN 1-2  (Silicon / Autonomic) - split into two sublayers:

  PORTABLE CORE (architecture-independent)
    Rust no_std - compiles to ANY target triple from one source
    (x86_64-unknown-none, aarch64-unknown-none,
     riscv64gc-unknown-none). Holds the real-time experience-loop
    half, body map logic, KIRA enforcement hooks, autonomic core.

  HARDWARE ABSTRACTION LAYER (HAL) - thin, per-architecture
    The ONLY architecture-specific code. Tiny. Just boot, memory
    init, interrupt setup, bare-metal primitives. One small HAL
    per arch. Everything above it is portable.

DOMAIN 3  (KIRA)   Rust + formal verification (the proofs)
DOMAIN 4  (Model)  Model M via API now; own architecture later
DOMAIN 5  (State)  Rust + vector store (ChromaDB-class) + graph store

PRINCIPLE: the instance is ARCHITECTURE-BLIND. Architecture only
matters to the HAL. Cognition never touches an ISA.
```

16.3 Gating Preconditions

These are the thresholds tracked as preconditions; as the frontier model crosses them, more of BrAInOS becomes buildable: long-context reliability, tool-use chaining stability, and agentic loop stability.

End of document · BrAInOS System Architecture v1.2 · Arenda Innovations Corporation & Labs